



Figure 4: JVisualVM threads pie chart

Threads that have a significant monitor time are a cause for concern, as they indicate bottlenecks in the code. A monitor is used to synchronise multiple threads and if the monitor time of threads is more, it indicates that they individually ran without any issue, but were constrained by an underlying resource or a data structure that was shared among all of them.

The tabular view gives the same information but aggregated over a period of time. In the tabular view, there is no way to correlate the running state of a Thread A against, say, the wait state of Thread B. It just lists the amount of time spent in each state and the total time since the thread was started. Further detailed information on the percentage of time spent in each state, along with the exact details on when the state transitions happened, can be obtained from the *Details* tab. In this tab, general information on the amount of time spent can be obtained in the *General* tab. In the *Details* tab, the exact moments in time when the state transitions occurred can be obtained.

In addition to the above information, a thread dump can be taken by clicking the *Thread Dump* button for offline analysis of the threads.

An important feature of thread dump analysis tools, including *jvisualvm*, is the ability to detect deadlocks automatically. As explained in the section on *jstack*, these tools can identify the monitors held by threads and analyse if there are deadlocks in the program. When *jvisualvm* is used to profile a running application, it detects the deadlock when it occurs. The thread dump taken from *jvisualvm* is the same as the dump generated by *jstack*.

Monitoring

Monitoring is the most commonly used feature in identifying performance (response time) bottlenecks and in spotting memory issues with applications. Both CPU and memory usage can be monitored. In case of the CPU, the amount of time spent in each method is reported and, hence, the method that is taking the maximum amount of time can be easily seen. In the case of the memory, the memory consumed by different objects in the program is reported, which shows the top memory consumers.

A couple of options are available for monitoring. The first one is *Sampler*, which as its name indicates, just samples the application periodically and gathers CPU and memory information about the running process. The second option available is *Profiler*, which modifies the byte code

of the running application to report the CPU and memory statistics more accurately. For instance, if *Sampler* runs with a frequency of three seconds, it just reports the information gathered at the three-second snapshot times. On the other hand, a profiler modifies the byte code so that *classes* and *method* report the instrumentation information to an agent. Usually these agents implement interfaces in *java.lang.instrument* interface. This kind of monitoring has an additional overhead on the running program, but the results in terms of the exact number of bytes used by an object, or the percentage of CPU time spent in a method, are much more accurate than a sampler. If quick information is needed without much overhead to the running application or a lot of accuracy, a sampler can be used. If more accuracy is needed, use a profiler. Using a sampler is common in situations where we may need to quickly identify a bottleneck module, class or a huge memory consuming object. A profiler could be used to accurately identify the exact problem within that module or class, or in situations where the bottleneck cannot be identified using a plain sampler.

In JVisualVM, the CPU or memory can be monitored in either *Sampling* mode or in the *Profiling* mode using the *CPU* or *Memory* buttons.

Sampler

The sampler settings can be viewed by clicking the *Settings* check box. This shows the options available on CPU and memory sampling. A filter on packages can be specified to either include or exclude specific packages from sampling. In addition to this filter, options are also available to adjust the rate of sampling the application and the rate of displaying the results.

In the case of the memory, only the sampling rate can be adjusted. The filter on the CPU is very useful in application server environments as well. Application server related classes such as *com.ibm.** or *com.oracle.** can be filtered out and profiling can be performed only on the application code.

A CPU sampler gives the amount of time spent in various methods. The three values given out are as follows.

Self-time [%]: This is the percentage of the overall time spent in a particular method.

Self-time: The amount of time spent in a method. This does not include the time spent in the methods invoked from this method. So, this does not accumulate the time spent in subsequent methods. To drill down into a particular method and find out the time spent in subsequent method calls, a snapshot can be used.

Self-time (CPU): This is the estimated real CPU time spent in executing the method. Like the self-time mentioned above, this does not include the time spent in subsequently invoked methods.

The thread CPU time tab gives the same information mentioned above on a per-thread basis instead of a per-method basis.